



Figure 1: OS commanding

A valid URL for this might be `http://www.example.com/viewfiles.php?username=arpit`. If a semicolon is used in the input, it will mark the end of the first command, and the beginning of a second. An example of a malicious URL would be `http://www.example.com/viewfiles.php?username=arpit;cat%20/etc/passwd`, which again displays the contents of the `passwd` file.

With such a vulnerability, attackers can execute any binary on the server like starting a telnet server, logging in to it with the privileges of a Web server user, performing exploits to gain root access, and perhaps attacking other hosts that are reachable from the compromised server.

The time for security

OS commanding is on the list of least-addressed vulnerabilities, but the following security measures can help curb it considerably:

1. Remove the execution bit from the *everyone* group (`-rwxrwxrwx`) for OS commands. The Web server user account will not be able to execute the targeted command even if an attacker is able to trick the Web application into attempting to execute it.
2. Validate all input fields for characters like `;`, `|`, `'`, `/` or `%00`. Now, since the list of such characters can be endless, the best thing would be to only allow the expected input, and filter out everything else—create a white-list of acceptable inputs that strictly conform to specifications, and reject any other input, or transform it into acceptable input. For example, to neutralise the ampersand character (`&`) that appears in user data, and which needs to be a part of an HTML page, it must be converted to `&`.
3. As additional security, you can also try filtering out command directory names such as `bin`, `sbin`, `opt`.

Path-traversal (a.k.a. directory traversal) attacks

Web servers are generally set up to restrict public access to a specific portion of the server's file-system, typically called the 'Web document root' directory. This directory contains files and any scripts that provide Web application functionality. In a path-traversal attack, an intruder manipulates a URL in such a way that the Web server executes, or reveals the contents of, a file anywhere on the server—including outside the document root. Such attacks take advantage of special-character sequences in URL input parameters, cookies and

HTTP request headers. The most basic path traversal attack uses the `../` character sequence to alter the document or resource location requested in a URL. Although most Web servers prevent this method from escaping the Web document root, alternate encodings of the `../` sequence, such as Unicode-encoding, can bypass basic security filters. Even if a Web server properly restricts path-traversal attempts in the URL path, any application that exposes an HTTP-based interface is also potentially vulnerable to such attacks.



Note: For UNIX systems, the parent directory is `../` while in Windows it is `..\`.

Attack Scenario 1

The valid URL `http://www.example.com/scripts/database.php?report=quarter1.txt` is used to display a text file. However, manipulating it into a malicious URL like `http://www.example.com/scripts/database.php?report=../scripts/database.php%00txt` will force the PHP application to display the source code of the `database.php` file, treating it as a text file whose contents are to be displayed. The attacker uses the `../` sequence to traverse one directory above the 'current' directory, and enter the `/scripts` directory. The `%00` sequence is used both to bypass a simple file extension check, and to cut off the extension when the file is read and processed by PHP. This example highlights the critical importance of always checking and cleaning user-supplied input before allowing it to be processed.

Attack Scenario 2

The PHP code below accepts a username, and then opens a file specific to that username. It can be exploited by passing a username that causes it to refer to a different file:

```

$username = $_GET['user'];
$$filename = "/home/users/$username";
readfile($filename);

```

Let's suppose a normal URL is `www.example.com/profile.php?user=arpit.pdf`. If an attacker passes a changed query string to make a malicious URL like `www.example.com/profile.php?user=../etc/passwd` then PHP will read `/etc/passwd` and output that to the attacker.

Path-traversal attacks mostly target an application's file upload, download, and display functionalities, like those often found in work-flow applications (where users can share documents); in blogging and auction applications (when users upload images); and in informational applications (when users retrieve documents like e-books, technical manuals, and company reports).

In many cases, there may be security measures in place—filters for forward or backward slashes—but here too, attackers